Assignment 3: Unit Test Automation

Software Quality Engineering
Group members: Faizan and Usman
23 October 2026

Project and scope

The selected project is Validators, an open-source Python library under the MIT licence. Its validation functions have clear input rules, which makes it suitable for equivalence class partitioning (ECP) and boundary value analysis (BVA). A fixed copy of version 0.36.0 is included in the repository; the application source has not been modified.

The scope covers three modules: email, length and ip_address. The public functions are email(), length(), ipv4() and ipv6(). There are 20 cases per function, giving 80 independently collected test cases. Each function has two ten-case review allocations. The application is function-based; pytest classes are used to organize the tests.

Test design

Email cases separate ordinary mailboxes from malformed addresses, missing parts, invalid dots and unsupported host forms. The 64-character local-part boundary is checked at 63, 64 and 65 characters. Length cases use minimum 3 and maximum 5, testing lengths 2 through 6. Additional cases cover zero, equal and reversed limits, Unicode, whitespace and invalid input.

IPv4 cases check octet limits, format errors and prefix values around 0 and 32. IPv6 cases check segment values, segment counts and prefixes around 0 and 128. Both address functions include CIDR, strict-prefix and host-bit options. These checks need no DNS or network access.

Measured execution results

Run	Passed	Result
Ordered full suite	80	Pass
Two-worker full suite	80	Pass
Smoke	8	Pass
Boundary	37	Pass
Negative	46	Pass
Faizan / Usman allocations	40 / 40	Pass / Pass

Local verification used Python 3.12.14 on Windows. Counts above come from actual pytest JSON reports. 'Negative' means the input is expected to fail validation; the automated test passes when that failure is handled correctly.

Automation and configuration

A virtual environment isolates the dependencies. requirements.txt installs the test tools and the vendored application. pytest reads data/validation_cases.csv at collection time and parameterizes every row using its unique case ID. All 80 tests consume the external CSV, including the four or more external-data cases requested in the brief.

The assertions check the public return contract: valid input returns exactly True, while invalid input returns a false-valued ValidationError. Raise-mode cases require that exception to be raised. Failure metadata must retain the function name and original value. This is stronger than checking only whether a result is truthy.

Markers provide module, smoke, BVA, ECP, negative and member-allocation groups. In serial mode pytest-order sets email first, length second, IPv4 third and IPv6 fourth. Rows inside each function keep CSV order. In parallel mode pytest-xdist uses two workers; global completion order is unrestricted. Tests do not rely on earlier tests or shared application state.

Run python scripts/run_tests.py inside the virtual environment for the ordered suite, or add --parallel for two workers. Add --suite smoke, boundary, negative, email, length, ip, faizan or usman to choose a group. JSON, JUnit XML and text logs are produced for each run. GitHub Actions is configured for Python 3.11 and 3.12; its live result must be checked in the repository Actions tab.

Coverage and limitations

Selected module	Statements covered	Statement coverage
email.py	14 of 15	93.3%
length.py	10 of 10	100%
ip_address.py	33 of 34	97.1%

These are statement-coverage figures for the selected modules, not for the whole library. The tests sample meaningful partitions but do not exhaust all domain lengths, RFC options or address forms. Passing tests establish the expected behaviour for these cases; they do not prove the absence of defects.

Deliverables and member work

The repository includes the Excel test register, CSV resource, ECP/BVA notes, automation configuration, measured execution evidence and a captioned setup video. Each Excel row maps to a CSV ID and a pytest result. The source licence and upstream commit are retained for attribution and repeatability.

The initial package was prepared through Faizan's Git account. Usman was unavailable during preparation. His 40-case block is allocated for his review and must not be treated as evidence of his completed individual contribution. He should reproduce the setup, check his cases and commit his actual work before evaluation. Both members need to explain the testing approach.

References

Project: https://github.com/python-validators/validators
Snapshot: 22786cc6e7283a0fa452c179797e447f798db9f0
Detailed design: docs/test_design.md
Measured results: reports/verification.json